DATA WAREHOUSING AND DATA MINING

# Complete Jury 2 Implementation Report

Integrated Clinical & Operational Healthcare Intelligence Platform: Data Warehouse with AI-Driven Patient Readmission & Bed Capacity Prediction

## Jury 2 focus: Predictive Patient Risk Mining and Clinical Feature Profiling using Healthcare Data

| Team member name | Registration number |
| --- | --- |
| _____ | _____ |
| _____ | _____ |
| _____ | _____ |
| _____ | _____ |

Department: _____

Institution: _____

Faculty / Guide: _____

Verified implementation and evidence edition - 21 October 2026. Personal and institutional details are intentionally left blank for the team to complete.

# Contents


Contents 2

Abstract 3

Chapter 1 - Introduction 4

Chapter 2 - Literature and Technical Background 5

Chapter 3 - Dataset Description 6

Chapter 4 - System Architecture 7

Chapter 5 - Jury 1 Implementation 8

Chapter 6 - Pattern Matching Implementation 10

Chapter 7 - Classification Model Comparison 12

Chapter 8 - Selected Best Classifier 14

Chapter 9 - Clinical Feature Profiling 17

Chapter 10 - Dashboard Implementation 20

Chapter 11 - Orange Data Mining Integration 23

Chapter 12 - Testing and Validation 25

Chapter 13 - Results and Discussion 27

Chapter 14 - Conclusion and Future Scope 28

References 29

Appendix A - Reproduction and artifact index 30

# Abstract

This implementation integrates a reproducible SQLite healthcare warehouse with three data-mining tasks: multiclass readmission prediction, diagnosis-medication association mining, and descriptive clinical feature profiling. It uses the preserved local UCI Diabetes 130-US Hospitals dataset, with 101,766 encounters and 71,518 patients. Jury 1 retains its ETL, SQL/OLAP and warehouse dashboard; Jury 2 adds a five-model benchmark, a saved inference pipeline and a six-view Streamlit demonstration.

Patients are separated across training, validation and test partitions to reduce leakage from repeated encounters. Among five fixed candidate configurations, Random Forest achieves the highest validation macro F1, 0.43471. After refitting on training plus validation data, it achieves test macro F1 0.43302, accuracy 0.51113 and macro one-vs-rest ROC-AUC 0.65031. Early-readmission recall is 0.30038 and precision is 0.19927, limiting practical reliability. Apriori and FP-Growth produce matching sets of 85 frequent itemsets and 23 filtered rules on all encounters. These describe associations rather than treatment effects.

The repository was backed up before restructuring, its source data were hash-verified, and 29 automated tests passed after repairs. Genuine browser evidence covers the three faculty-required dashboard views and saved-model inference. Supplementary Orange workflows use a separate 3,000-patient sample with explicit categorical data and saved widget settings. The project remains an academic discharge-time analytical prototype. Bed-capacity forecasting and length-of-stay prediction are future extensions.

# Chapter 1 - Introduction

## 1.1 Background and problem

Healthcare encounters combine demographic categories, diagnoses, treatments and operational measures. Analytical work must organise these fields consistently before attempting prediction or pattern discovery. A warehouse supports repeatable aggregation, while a mining layer tests whether historical encounter characteristics distinguish recorded outcomes and recurring diagnosis-medication combinations.

The project addresses two connected academic problems: constructing a queryable star schema and evaluating mining methods with an auditable path from source data to displayed evidence. Repeated patients, outcome imbalance and retrospective variables make this more than a simple accuracy exercise. A convincing demonstration must explain the data grain, prediction point, evaluation split and limitations of each result.

## 1.2 Objectives, USP and MVP

The objectives are to preserve the original cohort, provide repeatable ETL and OLAP, compare five classifiers consistently, select a model using validation evidence, verify Apriori and FP-Growth results, and display all three faculty-required views. A supplementary Orange demonstration makes categorical data and evaluation workflows inspectable without replacing the Python implementation.

The proposed distinguishing contribution is traceability across warehouse, feature definitions, model selection, rule mining and dashboard evidence in one small academic repository. The working MVP comprises the SQLite warehouse, precomputed analytical outputs, the six-view Streamlit application, a saved discharge-time inference pipeline, and the supplementary Orange workflows. It is not an operational hospital product.

## 1.3 Jury scope

The common source is `data/diabetes_130_us.csv`. Jury 1 transforms it into a SQLite star schema. Jury 2 reads the original encounter-level source directly so prior visits, medication information and diagnosis codes remain available. The analyses share feature definitions but have separate purposes: supervised classification predicts recorded readmission outcome; association mining describes diagnosis/medication co-occurrence; profiling summarises observed cohorts. Streamlit reads precomputed results and loads a fitted pipeline only for inference. Orange is a supplementary visual experiment.

There is no hospital connection, live hospital feed, deployed clinical service, bed-capacity model or length-of-stay prediction model. Hospital days are an observed discharge-time predictor and a descriptive profile. The project title describes the broader roadmap.

# Chapter 2 - Literature and Technical Background

## 2.1 Data warehousing and OLAP

A star schema places measured events in a central fact table and descriptive attributes in dimensions. The design makes the grain explicit and allows SQL to aggregate measures across categories. Roll-up aggregates detail to a coarser level; slice restricts the analysed population. This project demonstrates those operations in SQLite rather than claiming a distributed production warehouse.

ETL is also a reproducibility boundary. If extraction silently changes the source or loading targets a different database from the dashboard, subsequent results cannot be explained reliably. The repaired implementation reads the local source without overwriting it and uses one canonical active warehouse location.

## 2.2 Supervised classification and evaluation

Supervised classification learns relationships between predictor values and observed class labels. Decision trees partition the feature space; Naive Bayes combines feature likelihoods; nearest-neighbour methods compare similarity; bagged forests average many trees; boosting combines sequential corrections. The five implemented candidates represent different modelling assumptions rather than five cosmetic variants.

Validation selects between candidate configurations, while an untouched test partition estimates performance after selection. Because patients can recur, grouping by patient is essential. Accuracy, precision, recall, macro F1 and one-vs-rest AUC answer different questions. The scikit-learn documentation defines the implemented metrics [2]. The results chapters use the actual recorded values, including the weak minority-class performance.

## 2.3 Frequent patterns and descriptive profiling

A frequent itemset appears in a sufficient fraction of transactions. Apriori uses the downward-closure property to prune candidate sets; FP-Growth mines compressed conditional patterns. Association rules add a direction to co-occurrence and are described by support, confidence and lift [3]. A high confidence value can arise from a common consequent, making lift and prevalence important context.

Clinical profiling groups encounters and summarises observed outcomes. Such aggregation is useful for describing historical cohorts, but group differences can reflect confounding, coding and repeated observations. Neither mining rules nor profile rates establish medical causes. These distinctions guide the interpretations throughout this report.

# Chapter 3 - Dataset Description

## 3.1 Source and local file

The official source is UCI's Diabetes 130-US Hospitals for Years 1999-2008 dataset [1]. The local source is the project copy, not a newly downloaded or sampled replacement. Its byte-level SHA-256 is recorded in the provenance file and matches the pre-edit audit. Local validation finds 101,766 rows, 50 columns and 71,518 distinct patient identifiers. UCI's predictor count differs from the raw-column count because identifiers and targets have separate roles.

The source represents historical inpatient encounters involving diabetes, rather than a current hospital feed. The cohort and available variables constrain generalisation. No new personal identifiers or team identities were invented for this report.

| Outcome | Encounter count | Share |
|---|---|---|
| NO | 54,864 | 53.91% |
| >30 | 35,545 | 34.93% |
| <30 | 11,357 | 11.16% |

## 3.2 Features, target and preprocessing

Question marks and missing values become missing numeric values or categorical `Unknown`. Numeric medians and scaling are fitted inside the training pipeline; categorical values are one-hot encoded with unseen-category handling and minimum frequency 50. The source has empty A1C entries, so the inference UI uses `Unknown` for those observations. Detailed ICD-9 codes are mapped to broad categories; leading-zero numeric diagnoses and decimal diagnoses are supported. This grouping is a technical feature mapping, not a diagnostic service.

Because medication usage and total hospital days are known by discharge, predictions are discharge-time research estimates. There is no admission-time forecasting claim. Dataset outcome `NO` means no recorded readmission in the source definition, rather than a guarantee that a patient was never subsequently hospitalised elsewhere.

| Predictor type | Included variables |
|---|---|
| Numeric | time_in_hospital; num_lab_procedures; num_procedures; num_medications; number_outpatient; number_emergency; number_inpatient; number_diagnoses |
| Categorical | age; gender; race; admission_type_id; insulin; change; diabetesMed; A1Cresult; diag_1_group; diag_2_group |

## 3.3 Data-quality limits

The missing-value strategy is technically consistent but cannot reconstruct unrecorded clinical information. Empty A1C observations are treated as Unknown; they are not assumed to be normal tests. Grouped ICD-9 codes lose detail while reducing sparsity. Age bands and demographic categories limit granularity. No hospital-level or chronological holdout is available in the implemented benchmark, and population shift from this historical cohort remains unmeasured.

# Chapter 4 - System Architecture

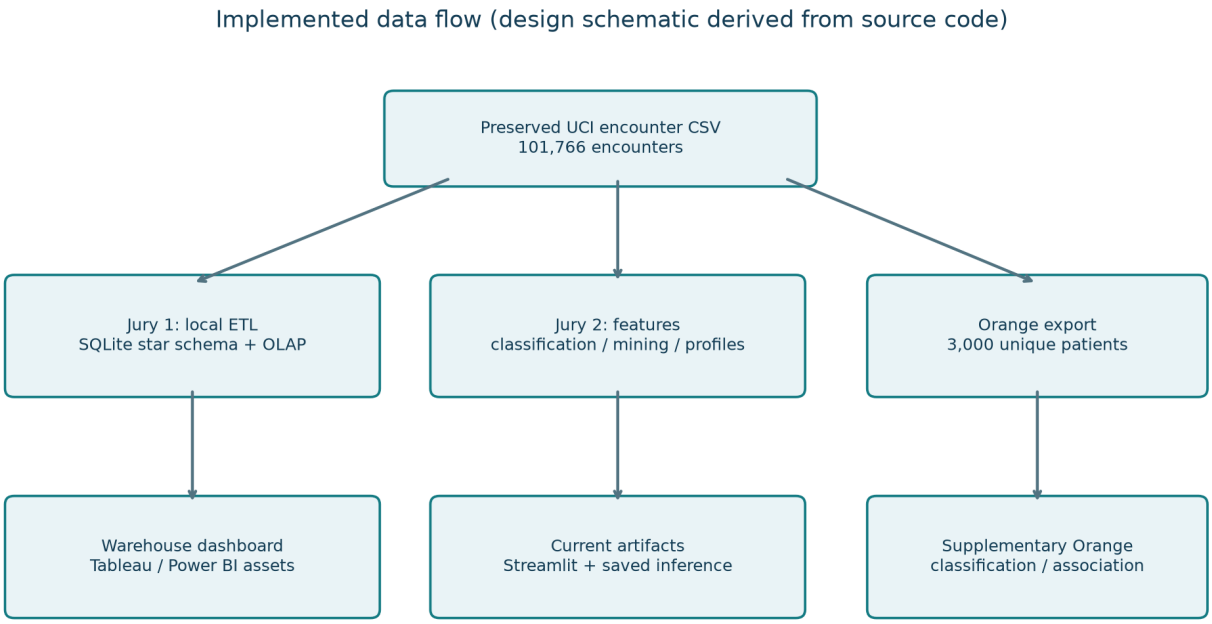



Figure 4.1. Design schematic derived from implemented modules and file flow; this is an explanatory diagram, not a runtime screenshot.

## 4.1 Data flow and phase integration

The original CSV is the common input. Jury 1 builds the fact and dimension tables for SQL/OLAP and warehouse visualisation. Jury 2 reads encounter-level columns from that same preserved CSV because its treatment/history predictors are not all stored in the existing fact table. This is a documented shared-source integration, not a claim that model training queries the warehouse.

Analysis scripts write versioned-by-reproduction output files in jury_2_dw/outputs. Streamlit displays these CSV/JSON/chart artifacts and loads the fitted pipeline for an example inference form. Orange preparation derives a distinct unique-patient demonstration sample. Keeping those experiment boundaries explicit avoids mixing Orange cross-validation scores with the primary holdout benchmark.

| Technology | Implemented role |
|---|---|
| Python / pandas / NumPy | Data loading, feature construction, ETL and analysis |
| SQLite / SQL | Star-schema warehouse, joins, roll-up and slice |
| scikit-learn / XGBoost | Five fixed classifiers, pipelines and evaluation |
| mlxtend | Apriori, FP-Growth, transaction encoding and rule metrics |
| Streamlit / Plotly / Matplotlib | Interactive views and reproducible analytical charts |
| Orange / Orange3-Associate | Supplementary categorical mining and classification workflows |
| pytest / Playwright / ReportLab | Automated checks, browser evidence and academic PDFs |

# Chapter 5 - Jury 1 Implementation

## 5.1 ETL, schema and SQLite

The fact grain is one source encounter. `Fact_Admissions` holds source encounter ID, patient key, generated admission and diagnosis keys, utilisation measures and readmission outcome. `Dim_Patient` has one record per patient; `Dim_Admission` has one record per admission/discharge/source combination; `Dim_Diagnosis` has one record per primary/secondary diagnosis and specialty combination. Pandas merges use many-to-one validation. Unique indexes detect duplicate keys; the validation suite checks the full fact-to-dimension join count.

The ETL previously fetched data and overwrote the local CSV, replaced original encounter IDs, and wrote a root-level database while the dashboard opened the `data/` database. It now reads the preserved local CSV and all active scripts agree on `data/healthcare_dw.db`. The historical root warehouse is in the local archive and external backup. The dimension design does not preserve changes to patient demographics over encounters. Explain this as a warehouse limitation rather than claiming a full clinical enterprise warehouse.

| Table | Verified row count |
|---|---|
| Fact_Admissions | 101,766 |
| Dim_Patient | 71,518 |
| Dim_Admission | 474 |
| Dim_Diagnosis | 34,387 |

## 5.2 SQL and visualisation

The roll-up query joins facts to patient demographics, groups by age band and computes admission count and mean stay. The slice query selects medication count greater than 15 and summarises outcomes and average laboratory count. Both were executed after ETL repair, and the console results are retained in deliverables/logs/olap.log.

The original Tableau workbook, Power BI asset and Jury 1 screenshots remain available. Their desktop applications were not retested in this run. The current Streamlit warehouse dashboard was executed with actual SQLite data and checked by AppTest and a real browser rendering. Figure 5.1 is a new genuine capture of that working dashboard.

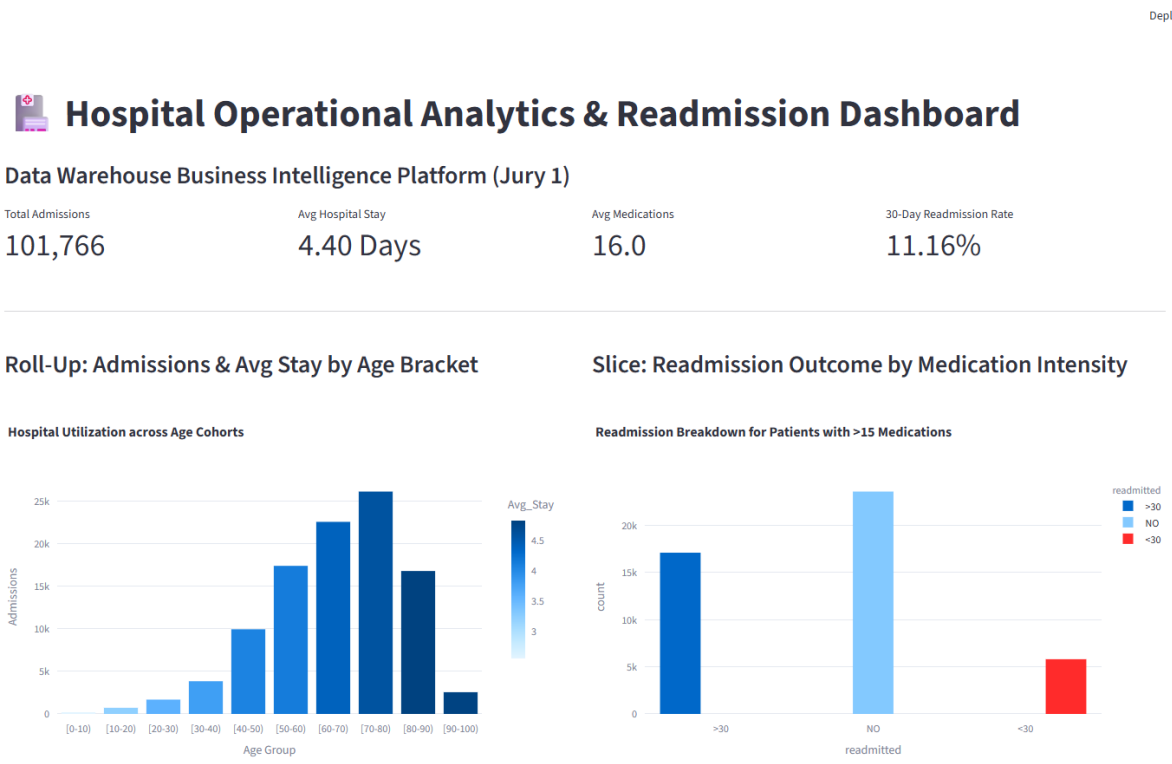



Figure 5.1. Genuine browser screenshot of the preserved Jury 1 warehouse dashboard.

# Chapter 6 - Pattern Matching Implementation

## 6.1 Transactions and algorithms

Each basket represents one encounter. Broad categories from all three diagnosis columns are deduplicated into `DIAG:category` items. Medications with `Steady`, `Up` or `Down` status become `MED:name` items. Absent medications and identifiers are excluded. The Boolean transaction matrix has 33 observed items.

Both algorithms use support 0.03 and maximum itemset size 3. Rules require confidence at least 0.35 and lift at least 1, and their combined sides must contain both a diagnosis and a medication. This filter allows a mixture of item types on either side; it is not restricted to one diagnosis implying one medication. Each algorithm produces 85 itemsets and 23 filtered rules. The test suite compares complete sorted itemset/rule tables and independently recomputes support, confidence and lift from the original baskets.

Support is P(A and B); confidence is P(B|A); lift is P(B|A)/P(B). A high confidence rule can simply reflect a very common consequent. Lift slightly above 1 is weak enrichment, not a strong medical discovery. The top retained rule has support 4.34%, confidence 65.67% and lift 1.229 for Diabetes plus Endocrine diagnoses implying insulin exposure. Disease severity, prescribing practice and repeated encounters can confound these associations. Neither medication efficacy nor treatment recommendations follow from them.

Apriori enumerates candidate itemsets using downward closure. FP-Growth compresses transactions and recursively grows frequent patterns. On this small item universe, Apriori was faster in this run; FP-Growth is not guaranteed to be faster. Compare outputs and disclose the measured workload rather than claiming universal speed.

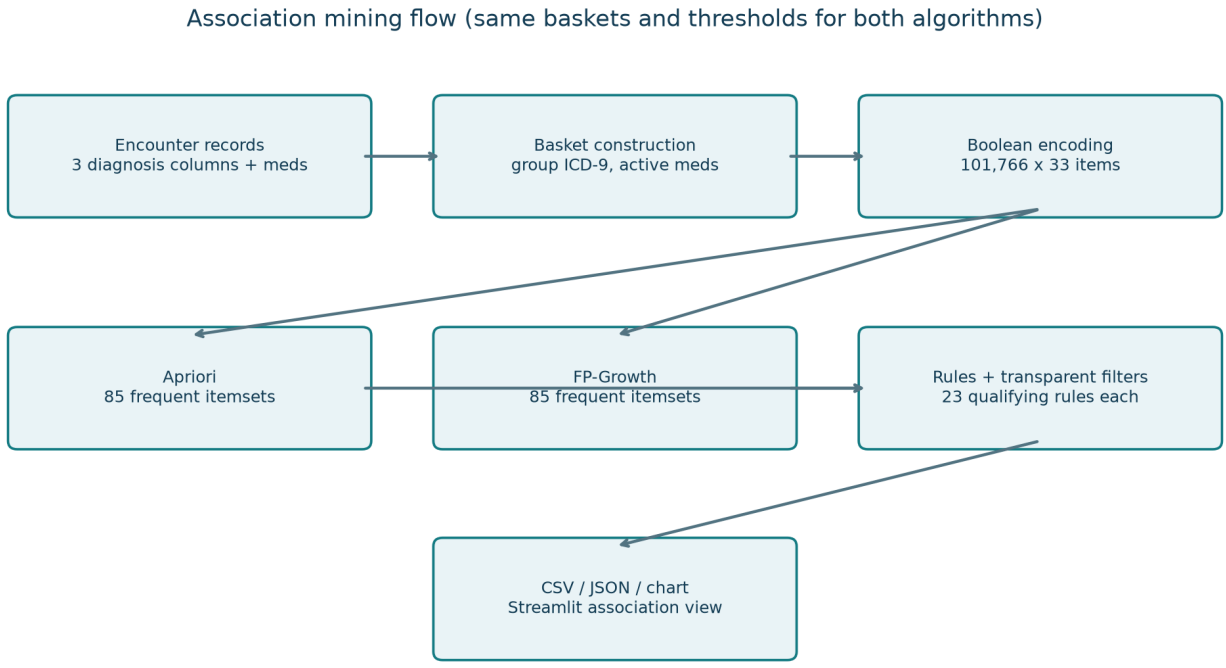



Figure 6.1. Implemented association-mining workflow, derived from the actual script.

## 6.2 Metric definitions and outputs

For antecedent A and consequent B, support is the count containing A union B divided by all encounters. Confidence is support(A union B) / support(A). Lift is confidence / support(B). These calculations use the same basket population and are independently recomputed in the tests for a real retained rule. CSV and JSON outputs contain the measured rule values.

Apriori took 0.089 seconds and FP-Growth took 0.256 seconds for itemset discovery in this run. These timings exclude surrounding load/export work and do not establish general speed rankings. The workload has only 33 observed items and a maximum itemset length of three, which makes candidate enumeration small.

| Method | Itemsets | Rules | Itemset time |
|---|---|---|---|
| Apriori | 85 | 23 | 0.089 s |
| FP-Growth | 85 | 23 | 0.256 s |

| Antecedent -> consequent | Support | Confidence | Lift |
|---|---|---|---|
| DIAG:Diabetes \| DIAG:Endocrine -> MED:insulin | 0.0434 | 0.6567 | 1.229 |
| MED:insulin \| MED:metformin -> DIAG:Diabetes | 0.0435 | 0.4421 | 1.183 |
| DIAG:Diabetes \| DIAG:Respiratory -> MED:insulin | 0.0323 | 0.6204 | 1.161 |
| MED:metformin -> DIAG:Diabetes | 0.0840 | 0.4279 | 1.145 |
| DIAG:Skin -> MED:insulin | 0.0441 | 0.6114 | 1.144 |

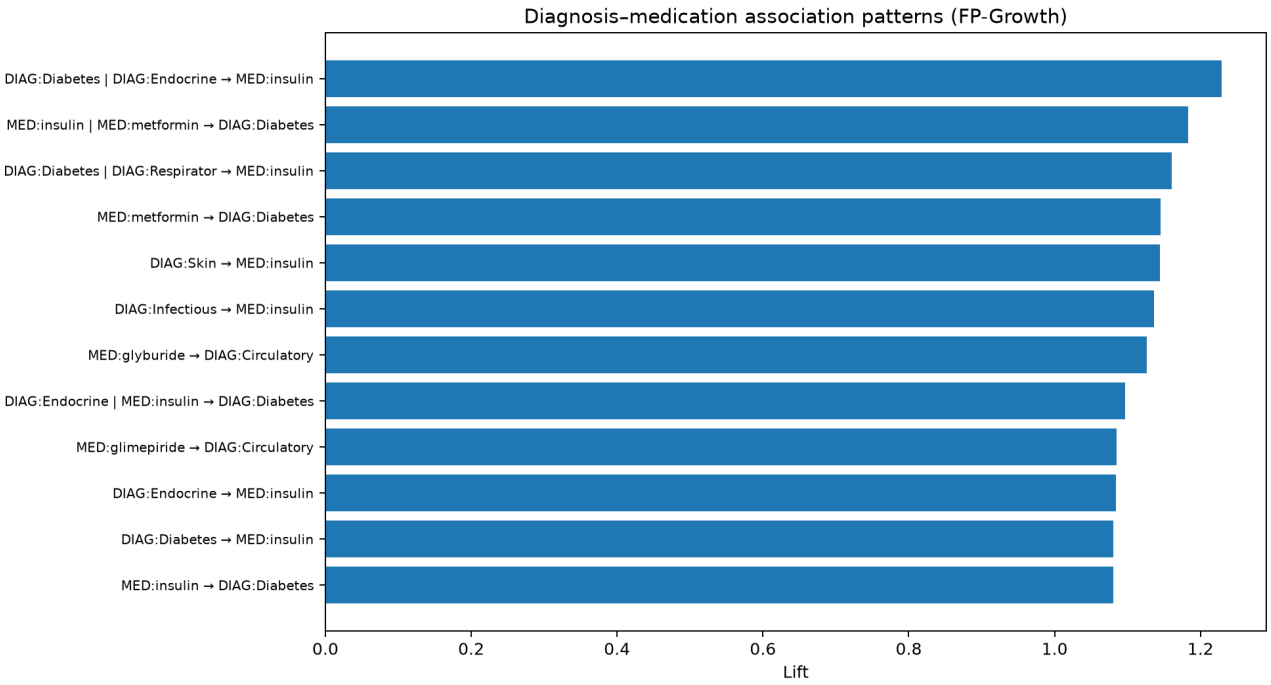



Figure 6.2. Actual highest-lift qualifying FP-Growth rules. Truncated plot labels are expanded in the saved rule table.

# Chapter 7 - Classification Model Comparison

## 7.1 Algorithms and evaluation process

GroupShuffleSplit reserves approximately 20% of patients for test using seed 42, then reserves approximately 25% of the remaining patients for validation using seed 43. Resulting encounter counts are 60,878 / 20,400 / 20,488. Groups are disjoint across partitions. Preprocessing is fit only on training data for comparisons. The same samples and feature definitions are used for all five models. Four algorithms receive balanced sample weights; kNN uses its distance-weighted neighbourhood and has no equivalent sample-weight interface. This difference is explicit.

Decision Tree uses depth 12 and minimum leaf size 30. Gaussian Naive Bayes models class-conditional independent Gaussian features. kNN uses 31 distance-weighted neighbours after scaling. Random Forest uses 140 trees, depth 18 and minimum leaf size 4. XGBoost uses 150 boosting rounds, depth 5, learning rate 0.07, subsampling and histogram trees. There was no broad hyperparameter search; selection is among these fixed candidate configurations.

Validation macro F1 determines the winner, with under-30-day recall as tie-breaker. Macro F1 weights each class equally despite imbalance. Refit the winner on train plus validation; use the untouched test patients once to estimate final performance. The saved joblib bundle contains the complete pipeline, feature names, labels and prediction point. Class order is `NO`, `>30`, `<30`.

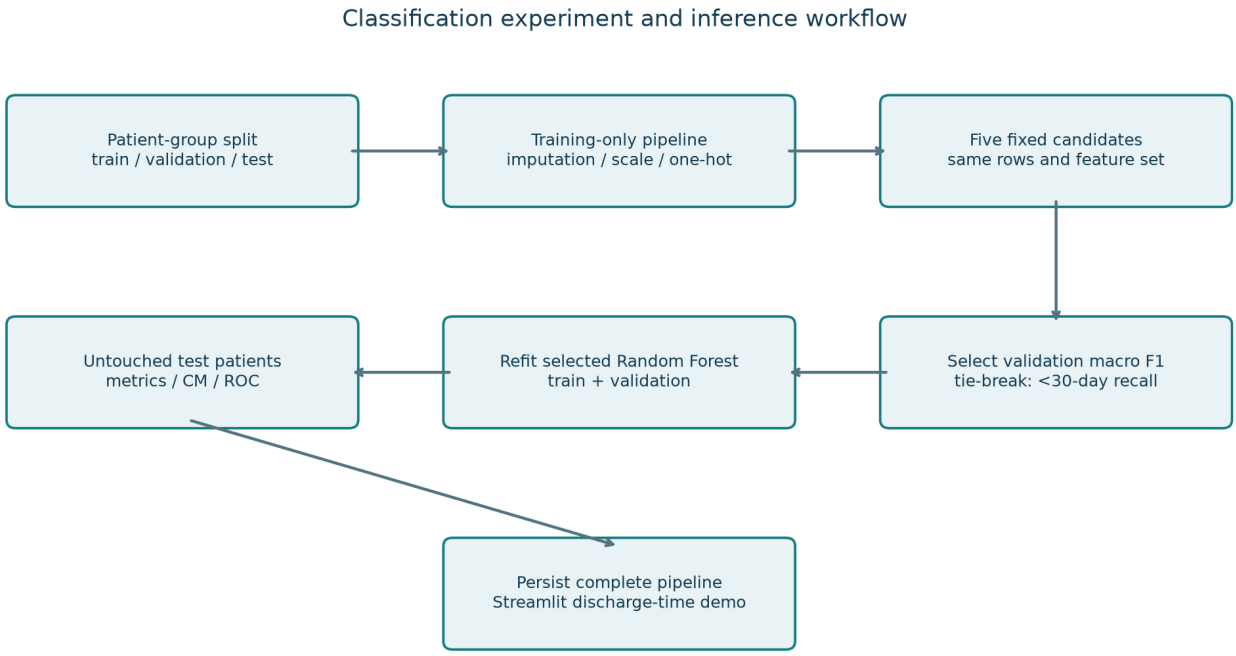



Figure 7.1. Training, validation selection, final test and saved inference workflow.

| Model | Accuracy | Macro F1 | <30 recall | Macro AUC |
|---|---|---|---|---|
| Random Forest | 0.5141 | 0.4347 | 0.3025 | 0.6462 |
| XGBoost | 0.4923 | 0.4238 | 0.3837 | 0.6457 |
| Decision Tree | 0.4346 | 0.3833 | 0.4367 | 0.6010 |
| KNN | 0.5628 | 0.3612 | 0.0153 | 0.6111 |
| Naive Bayes | 0.1432 | 0.1241 | 0.9358 | 0.5649 |

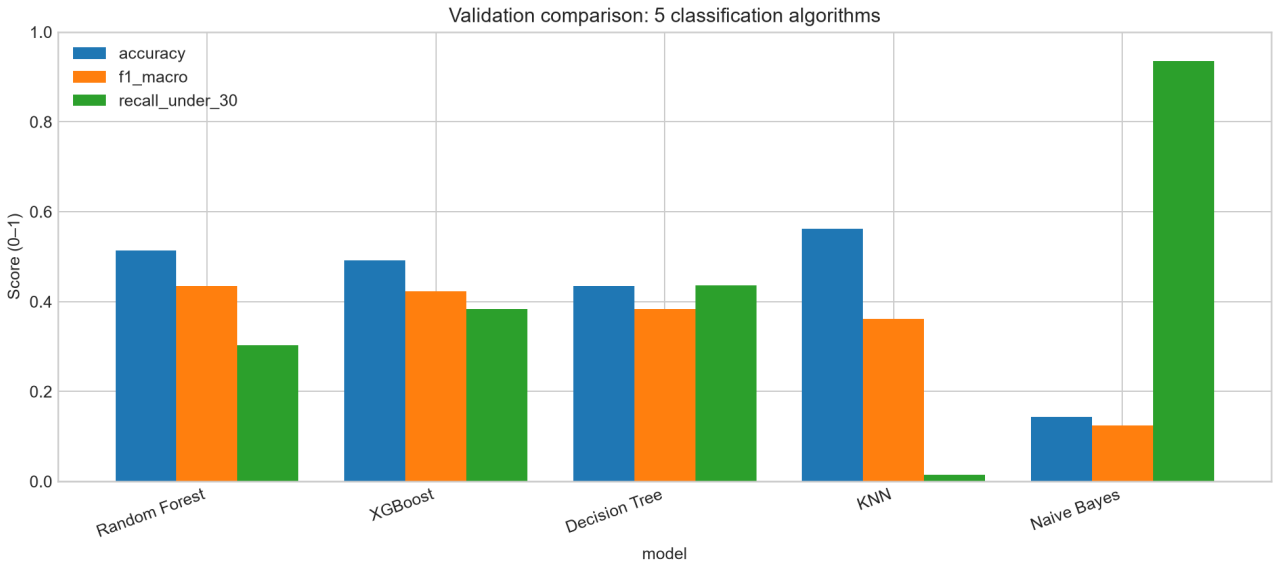



Figure 7.2. Actual validation comparison across the five fixed candidate configurations.

## 7.2 Selection rationale and weaknesses

Random Forest obtains validation macro F1 0.43471, ahead of XGBoost at 0.42379. XGBoost achieves higher early-readmission recall, while kNN achieves the highest accuracy but under-30-day recall only 0.01526. These differences illustrate that the winner depends on the declared objective. The chosen criterion weights all three outcome classes equally; it was not optimised for a clinical intervention cost.

Gaussian Naive Bayes performs poorly overall despite high early-readmission recall because it predicts that class too often. Its conditional independence and Gaussian likelihood assumptions fit these correlated one-hot features poorly. The benchmark retains this measured result rather than hiding it. No exhaustive hyperparameter tuning or confidence-interval study was performed, so close comparisons should not be treated as definitive statistical superiority.

# Chapter 8 - Selected Best Classifier

## 8.1 Random Forest design and final evaluation

The selected candidate is an ensemble of 140 decision trees with maximum depth 18 and minimum leaf size four. Balanced sample weights give rarer outcomes more training influence. Random feature selection and averaging reduce the variance of individual trees. The saved pipeline includes numeric median imputation and scaling, categorical encoding, and the fitted forest; inference uses the same transformations and fixed class order.

After selecting with validation macro F1, the forest is refit on 81,278 training-plus-validation encounters. Evaluation uses the 20,488 untouched encounters from 14,304 test patients. The output classes remain NO, >30 and <30. These are recorded readmission categories, and the model uses discharge-time information.

| Holdout metric | Measured value |
|---|---|
| accuracy | 0.51113 |
| precision_macro | 0.43288 |
| recall_macro | 0.44172 |
| f1_macro | 0.43302 |
| f1_weighted | 0.51712 |
| recall_under_30 | 0.30038 |
| precision_under_30 | 0.19927 |
| roc_auc_ovr_macro | 0.65031 |

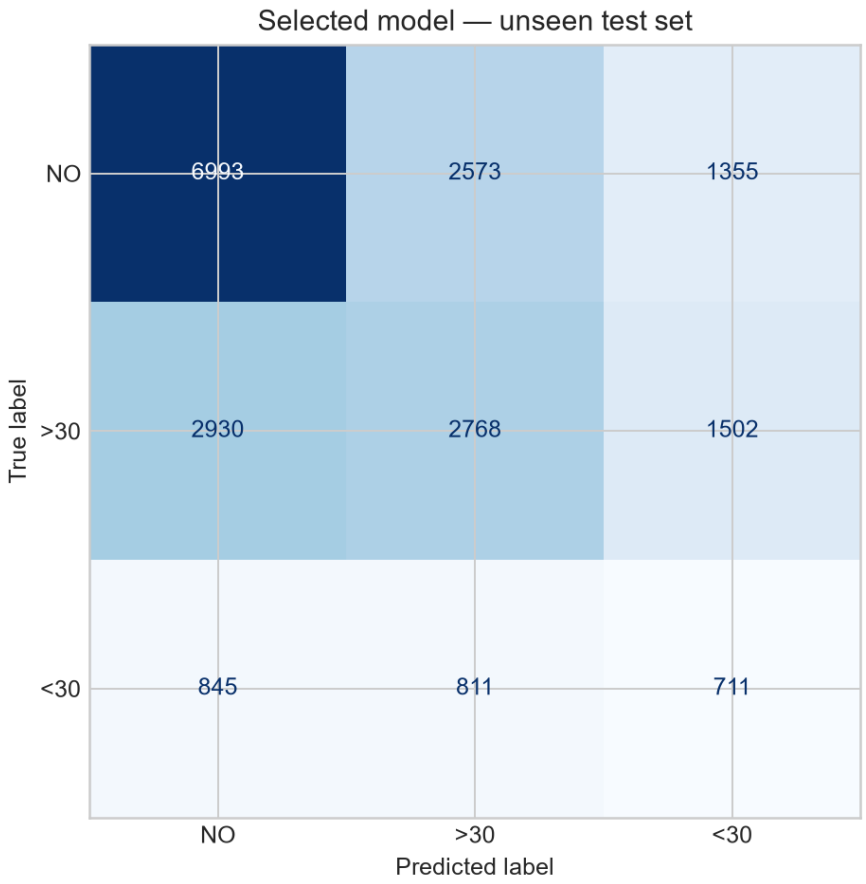



Figure 8.1. Actual confusion matrix on the untouched test set.

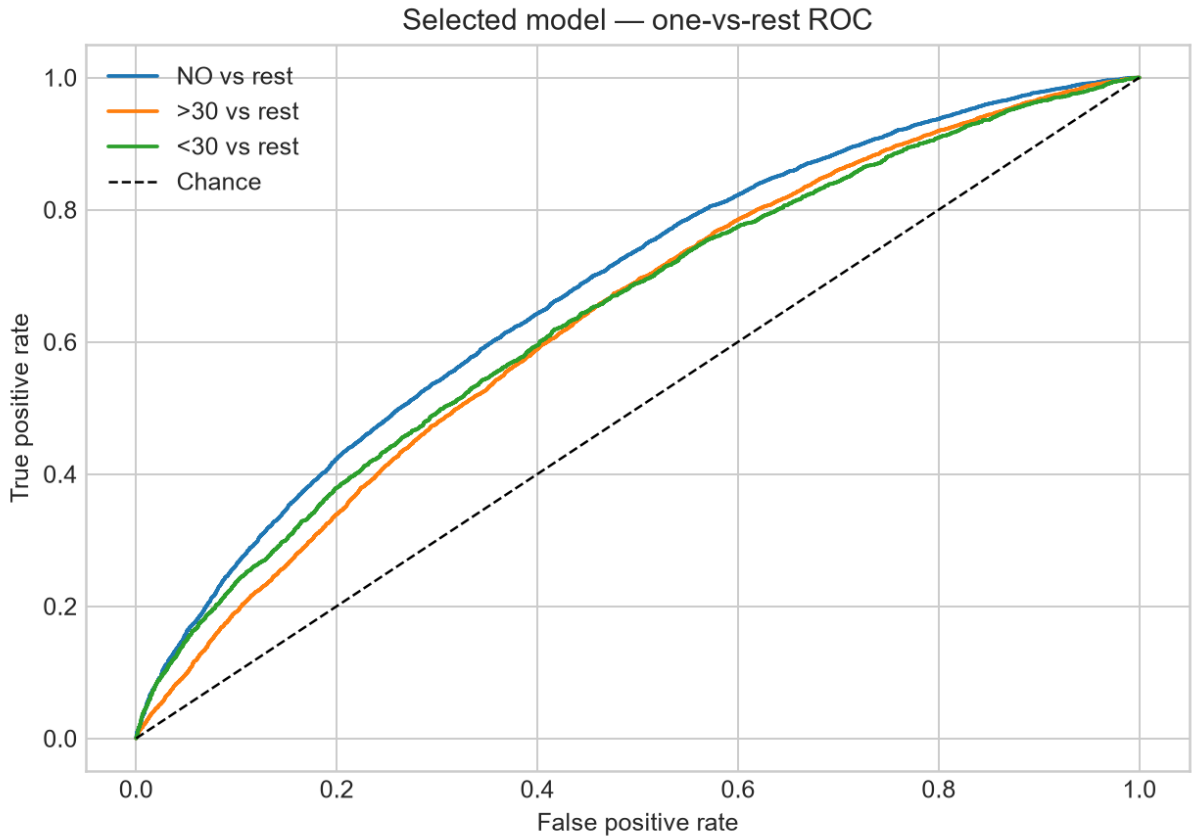



Figure 8.2. Actual one-vs-rest ROC curves on the untouched test set.

| Class | Precision | Recall | F1 | Support |
|---|---|---|---|---|
| NO | 0.6494 | 0.6403 | 0.6448 | 10921 |
| >30 | 0.4499 | 0.3844 | 0.4146 | 7200 |
| <30 | 0.1993 | 0.3004 | 0.2396 | 2367 |

## 8.2 Interpretation and practical limits

Only 711 of 2,367 true early-readmission encounters are correctly assigned to <30; 845 are assigned to NO and 811 to >30. Conversely, many predicted early readmissions are false positives, giving precision 0.19927. Such errors materially limit use beyond an academic demonstration.

An all-NO baseline on the same test set achieves accuracy 0.53304 and macro F1 0.23180. Random Forest has lower accuracy but substantially higher macro F1 and nonzero minority recall. This comparison explains the criterion without claiming the model is clinically useful. Macro AUC 0.65031 indicates modest ranking ability, not calibrated risk probabilities or benefit to patients.

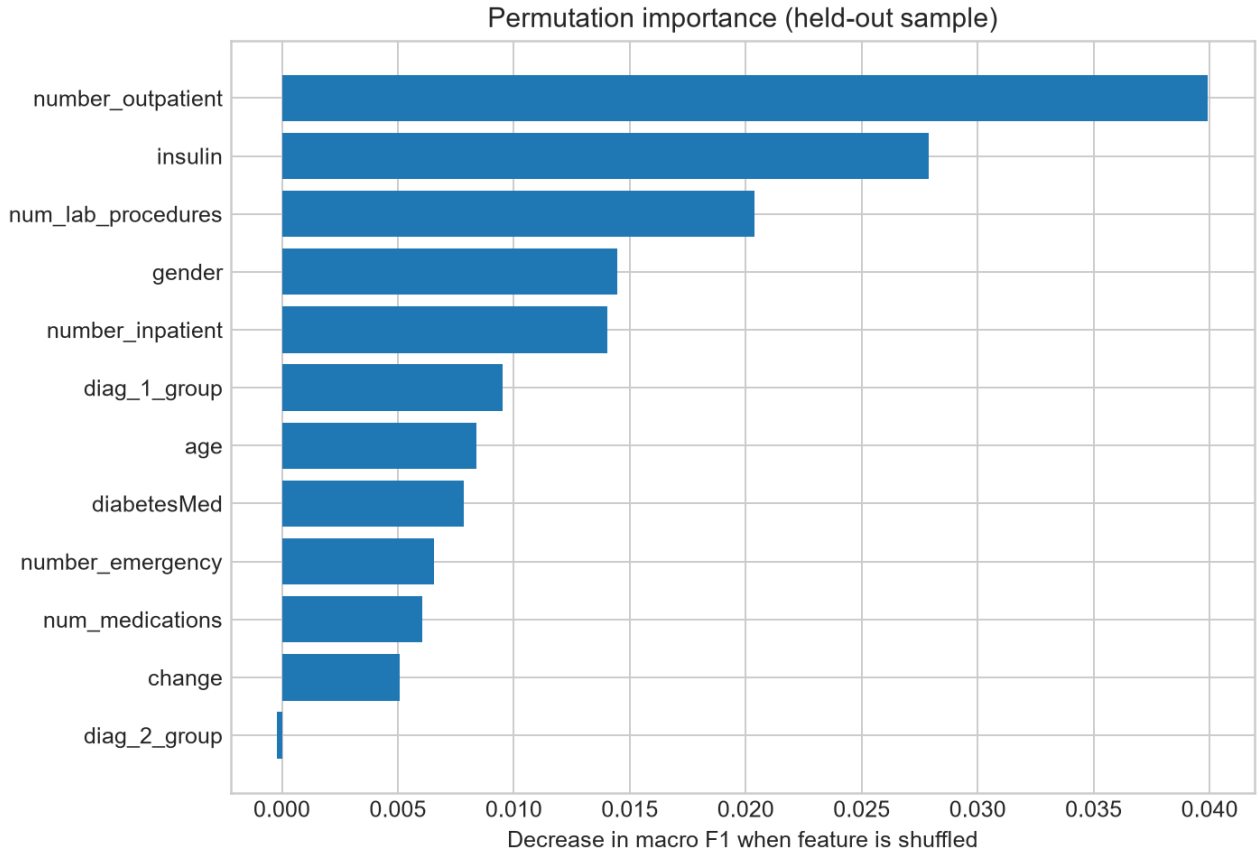



Figure 8.3. Exploratory permutation importance on 250 fixed test rows, two repeats.

## 8.3 Model interpretation

Permutation importance measures the fall in macro F1 when a feature is shuffled on a fixed 250-row test sample with two repeats. It is exploratory and noisy. Correlated predictors and small samples limit interpretation; negative importance can occur. It cannot establish causality or a clinical ranking of risk factors.

# Chapter 9 - Clinical Feature Profiling

## 9.1 Grouping and outputs

The profiling script summarises the same source cohort across age bands, primary diagnosis groups, medication counts, prior inpatient visits and observed length-of-stay bands. Each output contains encounter count, under-30-day count/rate, average hospital days and average medication count. Primary-diagnosis display excludes groups below 500 encounters to avoid presenting very small cohorts without context.

The overall early-readmission fraction is 11.16%, mean stay is 4.40 days and mean medication count is 16.02. Prior inpatient groups 0, 1 and 2+ have observed early-readmission rates 8.44%, 12.92% and 21.40%. This describes the historical sample, not a causal effect of prior care. Rates are encounter-weighted; repeated patients limit independence, and confidence intervals were not estimated.

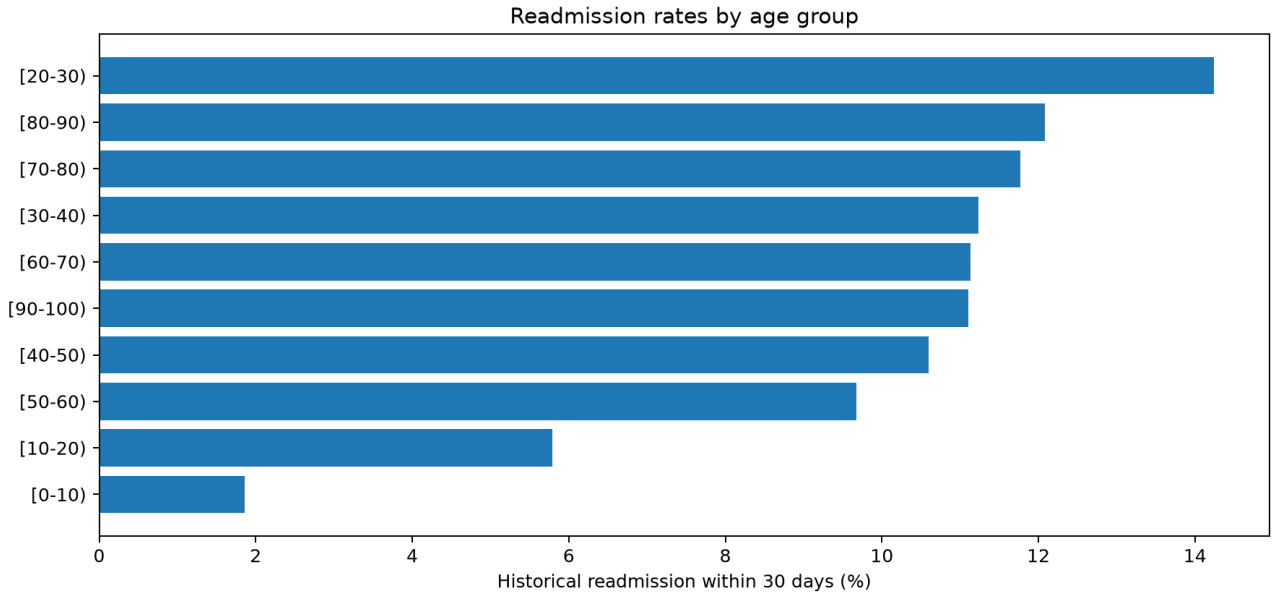



Figure 9.1. Actual descriptive readmission rates by age bands.

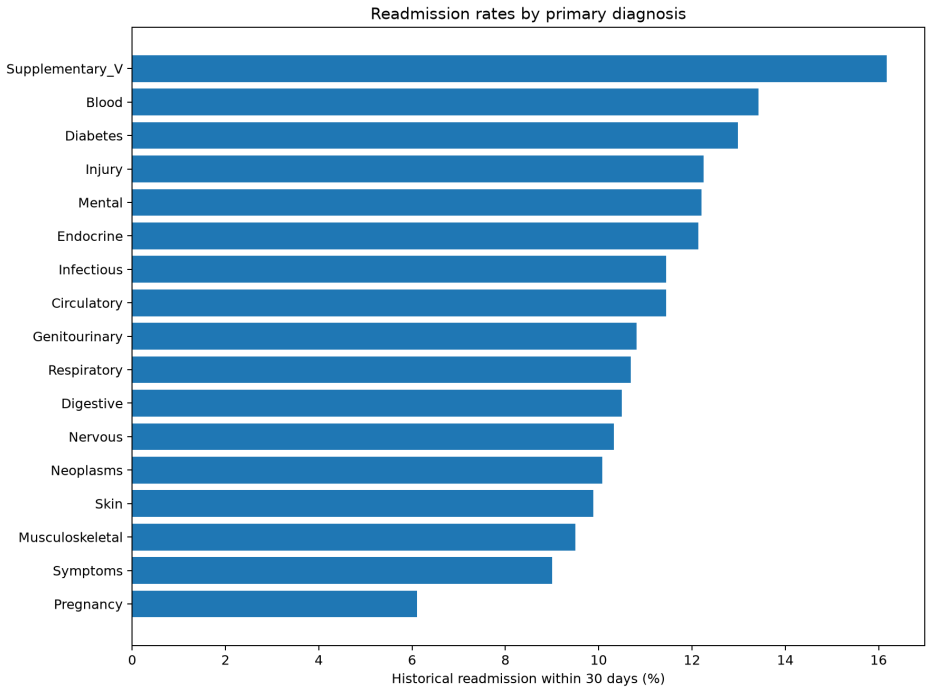



Figure 9.2. Actual descriptive readmission rates by primary diagnosis categories.

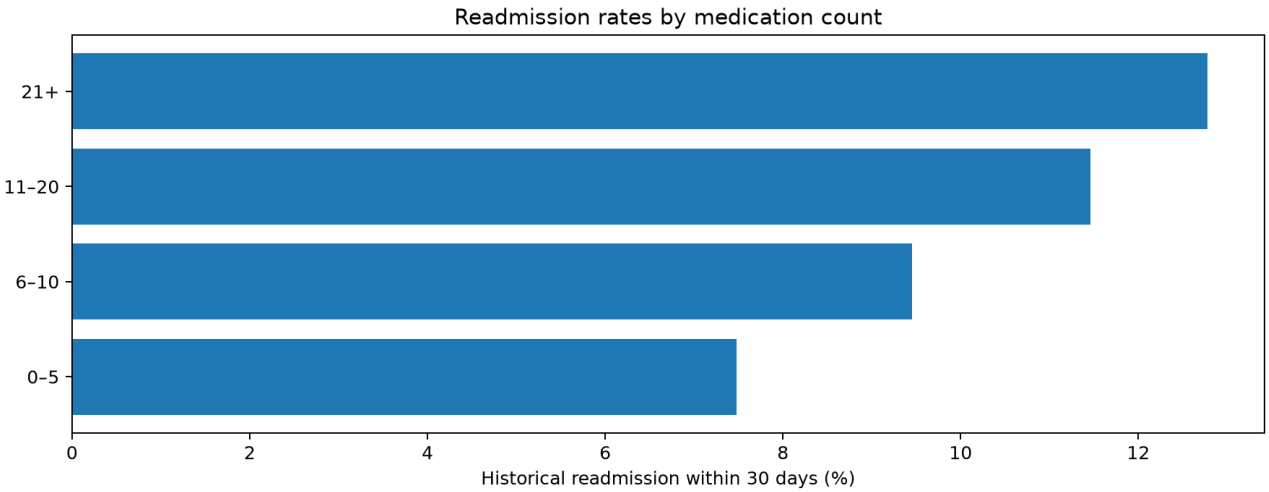



Figure 9.3. Actual descriptive readmission rates by medication-load bands.

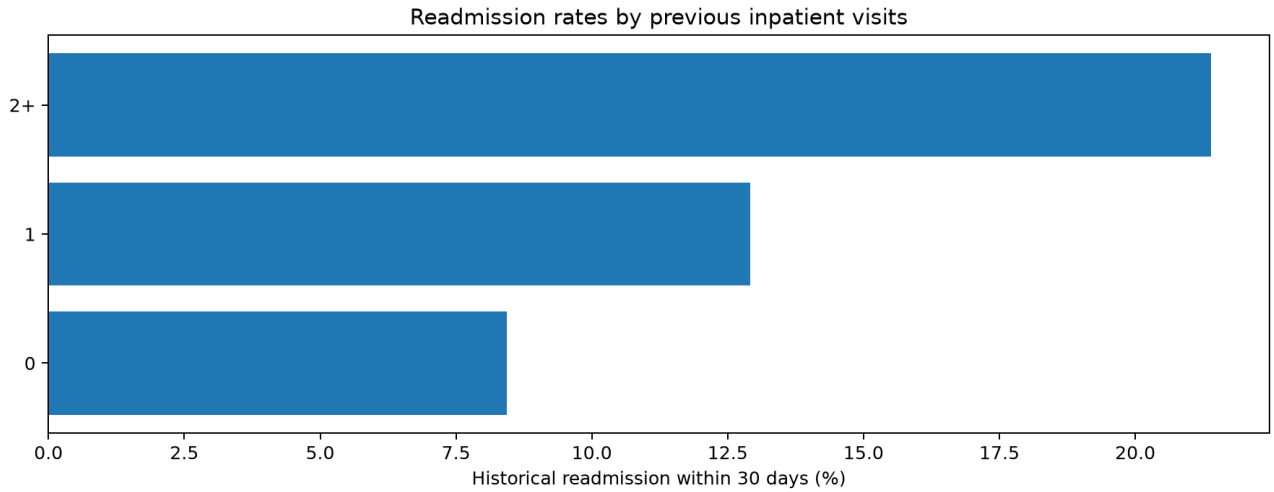



Figure 9.4. Actual descriptive readmission rates by prior inpatient visits.

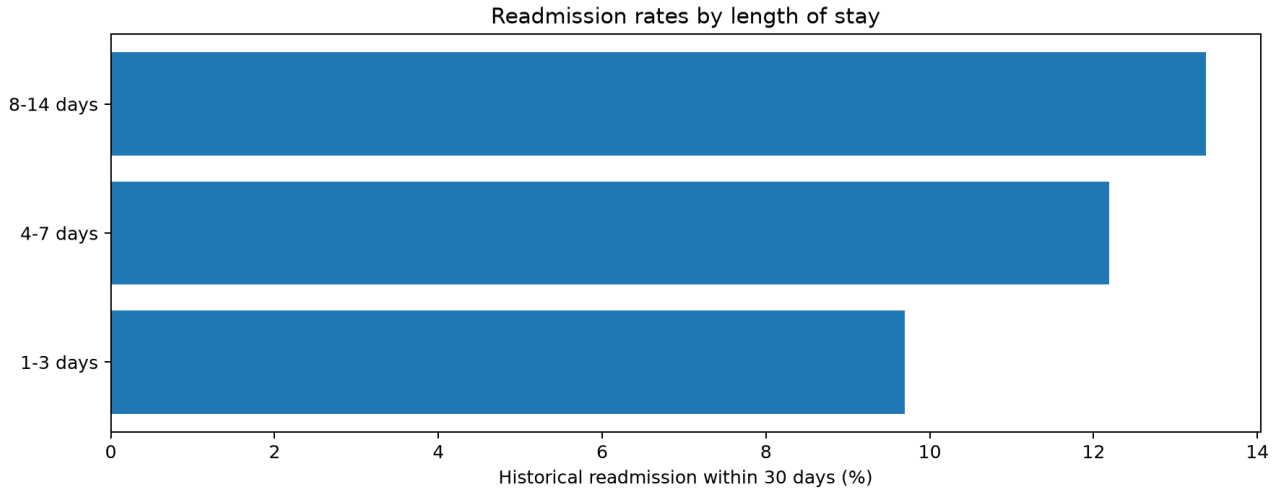



Figure 9.5. Actual descriptive readmission rates by observed length-of-stay bands.

# Chapter 10 - Dashboard Implementation

## 10.1 Application structure and faculty requirements

Streamlit's sidebar selects six views in one application. Association Mining exposes algorithm selection, frequent itemsets, a lift filter and rule metrics. Model Benchmark displays the five validation score rows, a selectable metric chart and selection rationale. Best Model displays the selected algorithm, test metrics, confusion matrix, ROC, class report and permutation importance. These are the three faculty-required views.

Overview provides context; Clinical Profiling allows cohort grouping; Live Risk Studio demonstrates the saved pipeline using a hypothetical bounded encounter form. The application does not retrain during browsing. It resolves artifacts relative to its source file, caches read-only results, checks the model bundle/class order, and reports missing or corrupt CSV/JSON files clearly.

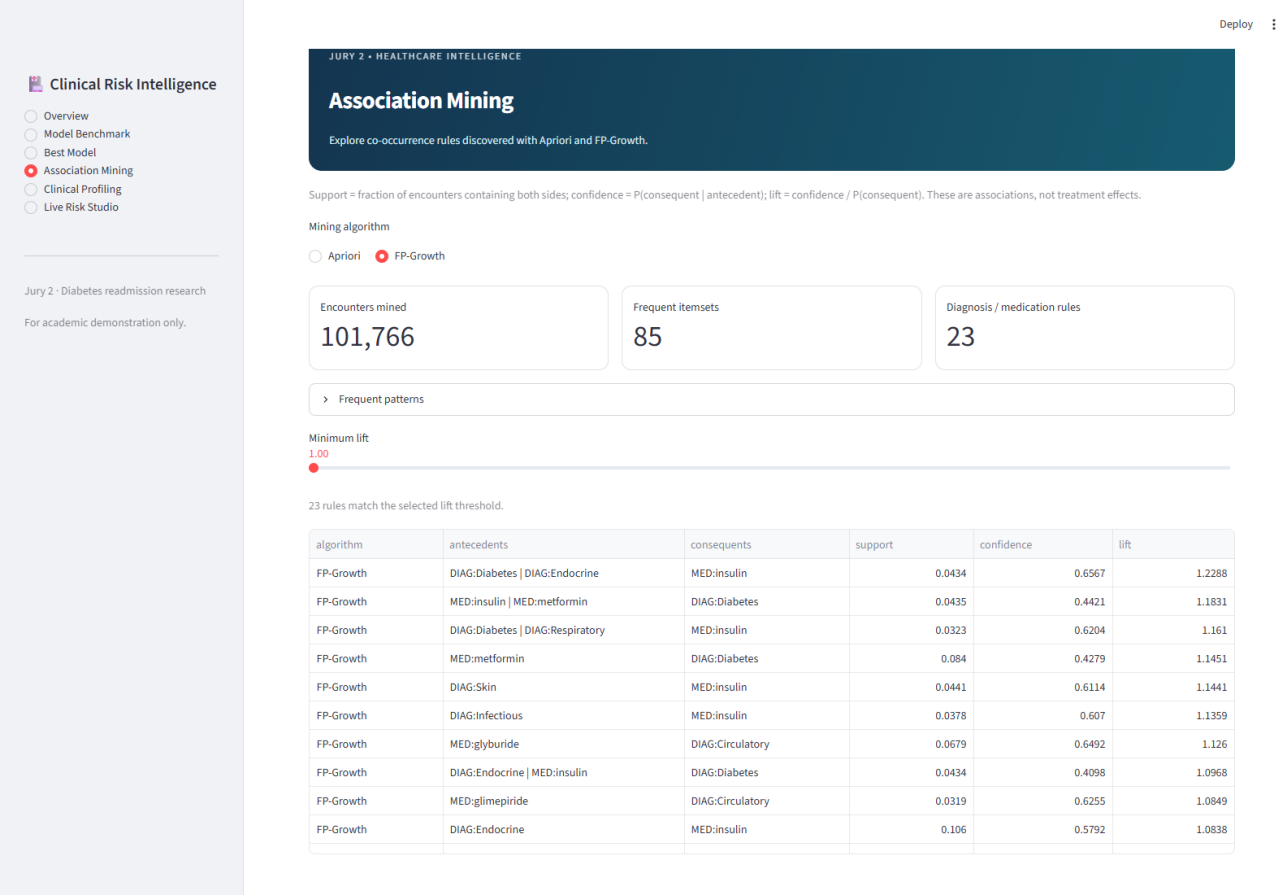
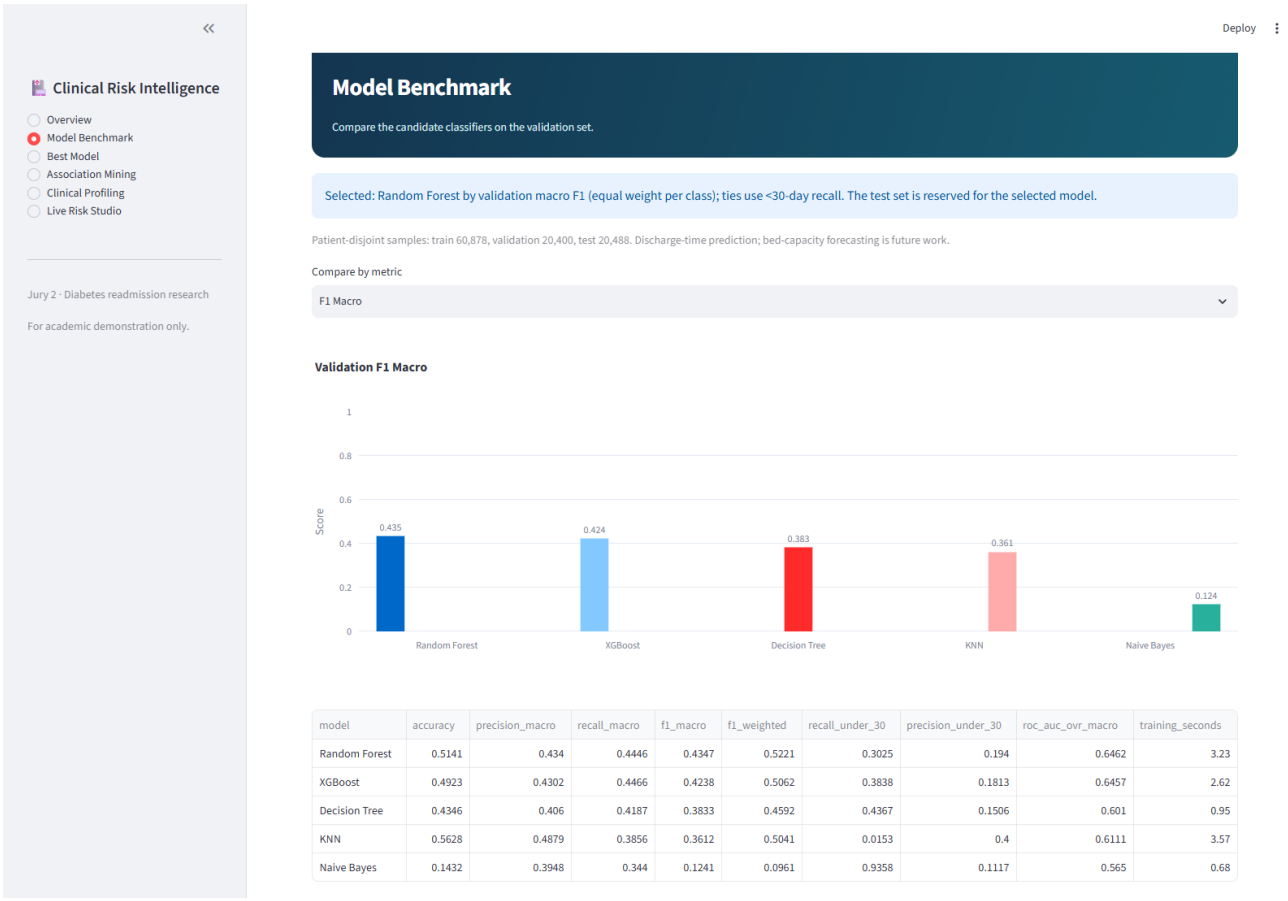
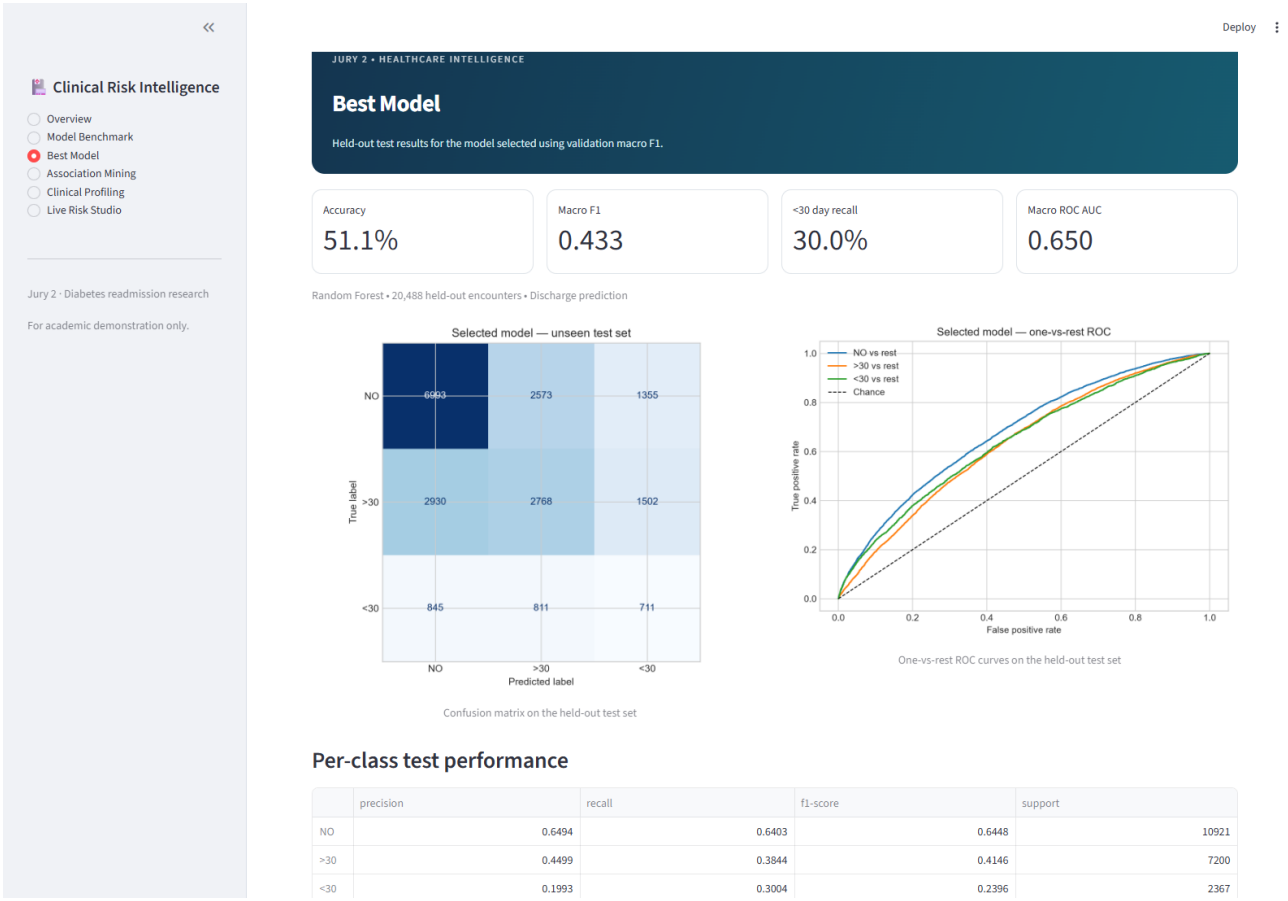



Figure 10.1. Genuine Streamlit browser screenshot: Association Mining, showing actual full-cohort rules.




| model | accuracy | precision_macro | recall_macro | f1_macro | f1_weighted | recall_under_30 | precision_under_30 | roc_auc_ovr_macro | training_seconds |
|---|---|---|---|---|---|---|---|---|---|
| Random Forest | 0.5141 | 0.434 | 0.4446 | 0.4347 | 0.5221 | 0.3025 | 0.194 | 0.6462 | 3.23 |
| XGBoost | 0.4923 | 0.4302 | 0.4466 | 0.4238 | 0.5062 | 0.3838 | 0.1813 | 0.6457 | 2.62 |
| Decision Tree | 0.4346 | 0.406 | 0.4187 | 0.3833 | 0.4592 | 0.4367 | 0.1506 | 0.601 | 0.95 |
| KNN | 0.5628 | 0.4879 | 0.3856 | 0.3612 | 0.5041 | 0.0153 | 0.4 | 0.6111 | 3.57 |
| Naive Bayes | 0.1432 | 0.3948 | 0.344 | 0.1241 | 0.0961 | 0.9358 | 0.1117 | 0.565 | 0.68 |

Figure 10.2. Genuine Streamlit browser screenshot: Five-model validation comparison and declared selection criterion.




## Per-class test performance

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| NO | 0.6494 | 0.6403 | 0.6448 | 10921 |
| >30 | 0.4499 | 0.3844 | 0.4146 | 7200 |
| <30 | 0.1993 | 0.3004 | 0.2396 | 2367 |

Figure 10.3. Genuine Streamlit browser screenshot: Selected Random Forest test metrics, confusion matrix and ROC.

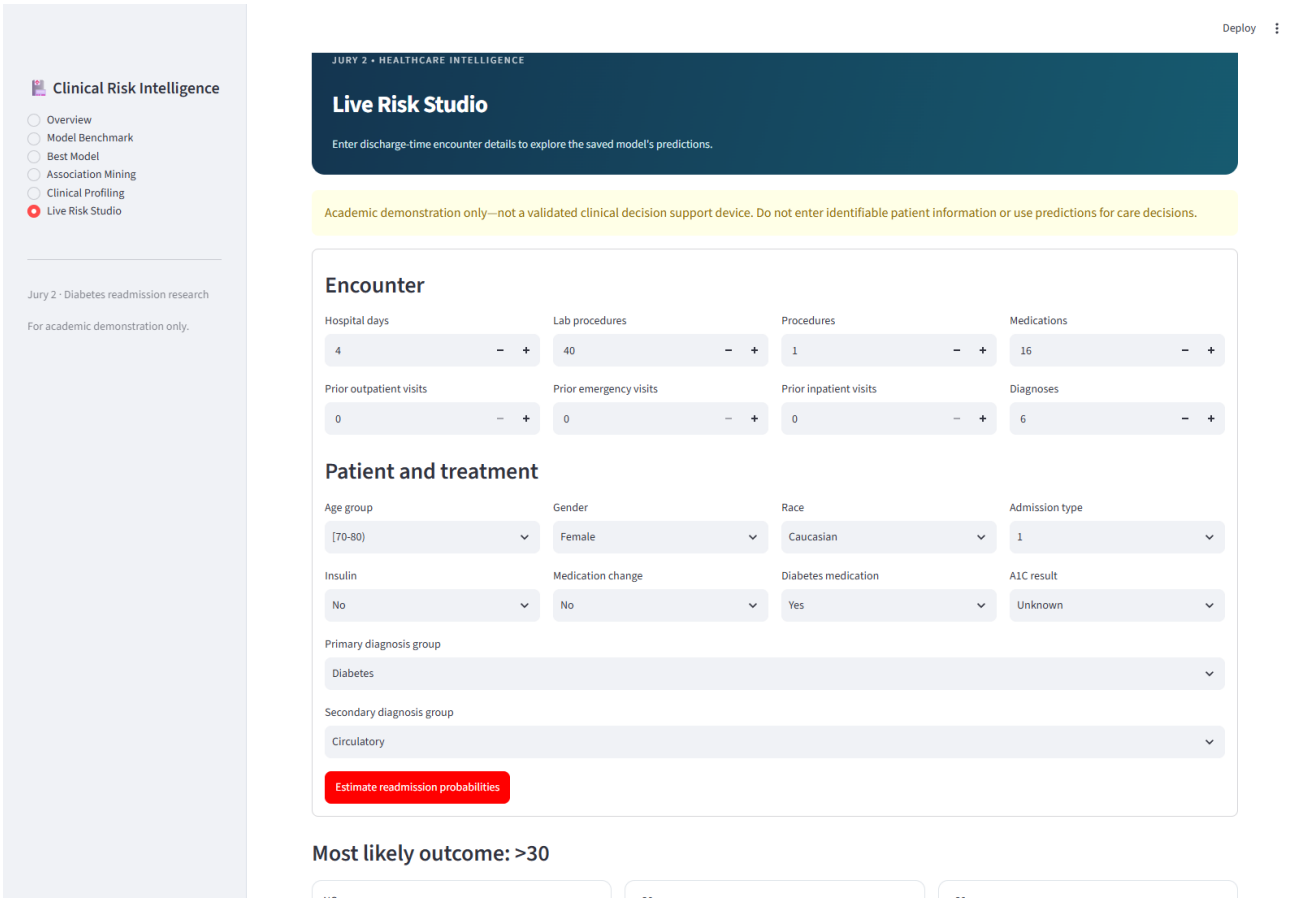



Figure 10.4. Genuine Streamlit browser screenshot: Working saved-pipeline example inference on hypothetical input.

## 10.2 Demonstration sequence

Begin with the historical source counts and discharge-time prediction scope. Open Association Mining and interpret one measured rule. Compare validation macro F1 across the five classifiers; explain kNN's accuracy/recall trade-off. Show the selected model's test confusion matrix and read the early-readmission row, then demonstrate a hypothetical inference. Use a clinical profile and supplementary Orange workflow only after the three required views. The separate evidence PDF provides larger screenshots for inspection.

# Chapter 11 - Orange Data Mining Integration

The original association `.tab` already contained 17 correctly marked discrete features. Installed Orange loaded all 3,000 rows and recognised all 17 inputs as discrete. The workflow XML had no saved File settings, permitting default Iris data to enter the mining widgets. Loading Iris reproduces the exact reported error. This confirms the failure mechanism; no screenshot of the original user's pre-edit File selection was available.

The repaired generator saves `RecentPath` settings with an absolute local path and a workflow-relative fallback. Regenerate workflows after moving the repository. Association inputs contain grouped diagnosis categories and active medication exposure. `?` means an absent item/missing value, so dominant non-use states are not mined. Patient and encounter IDs are excluded. Classification has 18 predictors and `readmitted` explicitly marked as its discrete class.

## Association workflow

1. Open `jury_2_dw/orange_demo/jury2_association.ows` in Orange.

2. Double-click Association data and confirm `orange_associations.tab`, 3,000 rows, 17 categorical inputs and no class.

3. Open Frequent Itemsets: support 3%, maximum displayed size 3. The verified sample gives 77 itemsets.

4. Open FP-Growth Rules: support 3%, confidence 35%, maximum two items on each rule side. The verified sample gives 45 rules, sent to Rule table as a genuine Orange Table.

5. Explain support, confidence and lift, and inspect both rule sides. Orange includes rules below lift 1; the primary Python output filters lift at least 1 and diagnosis/medication combinations.

The sample uses one first encounter per patient (ordered by encounter ID as a practical proxy), then samples 3,000 with seed 42. It does not represent an independently validated clinical cohort. Orange preserves primary/secondary/tertiary diagnosis positions, whereas the Python mining pipeline deduplicates diagnosis groups across positions. Rule counts therefore differ by design.

## Classification workflow

1. Open `jury_2_dw/orange_demo/jury2_classification.ows`.

2. Confirm `orange_classification.tab` in Classification data and `readmitted` as target.

3. Open Test & Score: five-fold stratified cross-validation with Decision Tree, Naive Bayes, kNN and Random Forest. Its four learners are supplementary; the primary benchmark has five.

4. Select a learner, then open Confusion Matrix and ROC Analysis. Switch the target class to `<30` for early readmission. Use Distributions to inspect cohort and class distributions.

5. Explain why these sample-based cross-validation scores differ from the full patient-group holdout benchmark.

The workflow explicitly disables the Tree widget's exhaustive binary categorical splits, which failed in the original desktop Test & Score check. It uses a maximum depth of 8, minimum leaf 10 and minimum internal size 20. kNN uses 15 neighbours; Random Forest uses 100 trees with repeatable training. Settings are loaded from the generated workflow in the automated Orange test so it exercises the demonstration configuration.

## Verification boundaries

`tests/verify_orange.py` checks workflow XML, imported widget classes, input/output channel names, data domains, real itemset/rule computation, the rule-table output, four-learner cross-validation, and Confusion Matrix/ROC/Distributions widget inputs. Counts, metrics and versions are saved to

`deliverables/logs/orange_validation.json`. Desktop evidence is stored in `deliverables/evidence`. Read the final validation record for exactly which desktop outputs were inspected. An upstream Orange3-Associate warning about deprecated deferred commit decoration is recorded; it does not prevent the verified rule output.

| Supplementary learner | 5-fold accuracy | Macro F1 | AUC |
|---|---|---|---|
| Decision Tree | 0.5253 | 0.3254 | 0.5182 |
| Naive Bayes | 0.5760 | 0.3713 | 0.6039 |
| KNN | 0.5553 | 0.3102 | 0.5254 |
| Random Forest | 0.5930 | 0.3151 | 0.5692 |

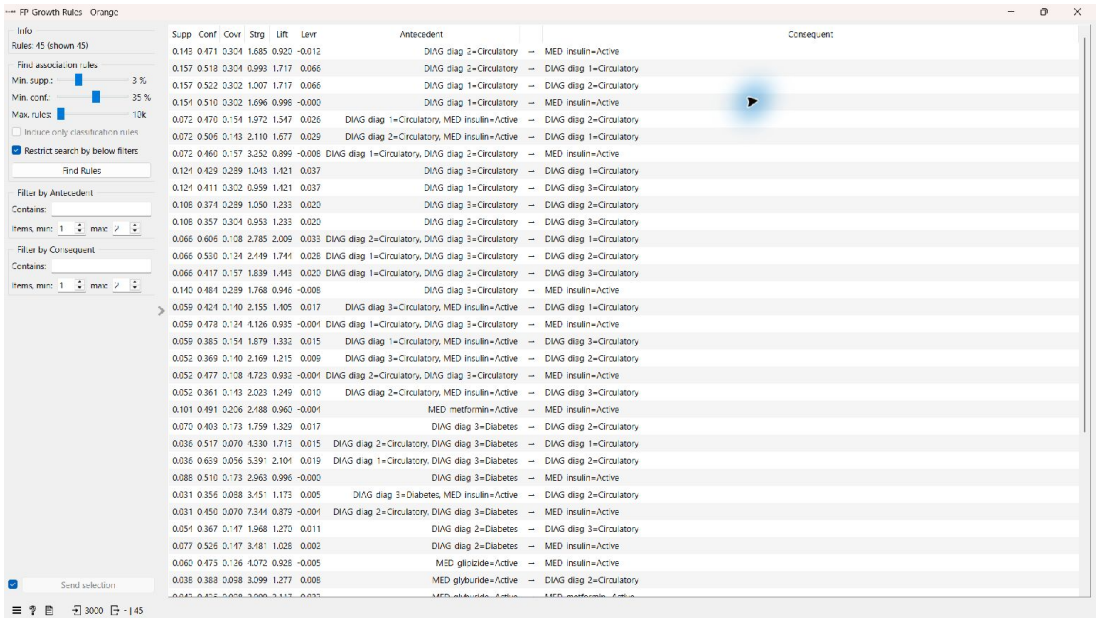



Figure 11.1. Genuine Orange screenshot. Desktop FP-Growth output: 45 actual rules at support 3% and confidence 35%.

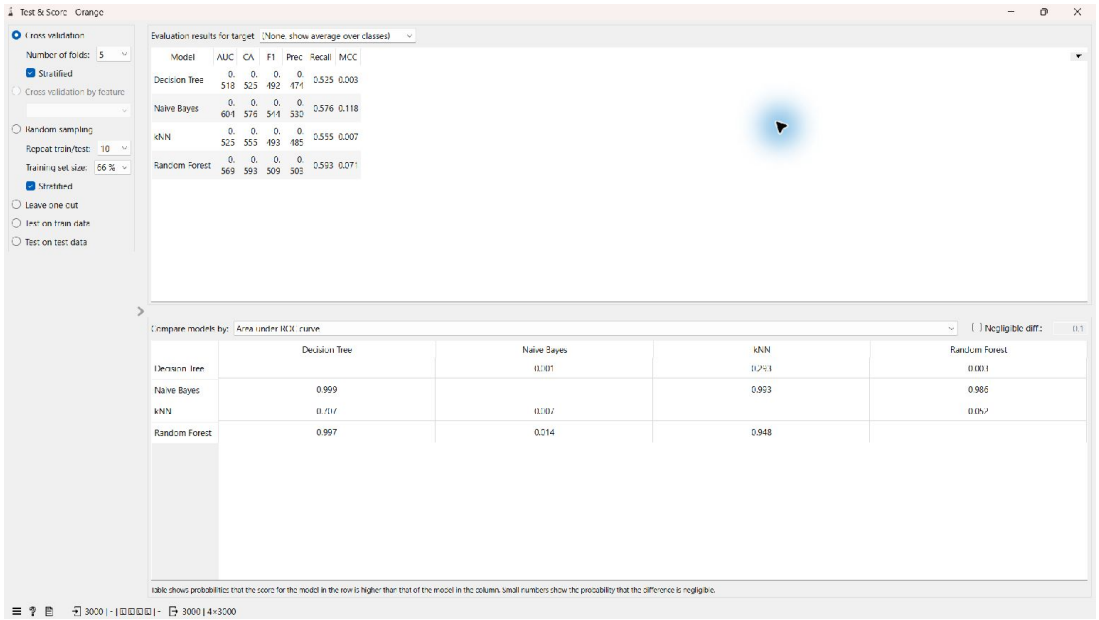



Figure 11.2. Genuine Orange screenshot. Desktop Test & Score after fixing the Tree settings; all four learners produce metrics.

# Chapter 12 - Testing and Validation

## Executed pipeline commands

Commands below ran in PowerShell from the repository root on 21 October 2026. `$orange` represents `C:\Users\offic\AppData\Local\Programs\Orange\python.exe`.

```
.\.venv\Scripts\python.exe jury_2_dw/orange_demo/prepare_orange_data.py

& $orange jury_2_dw/orange_demo/build_workflows.py

& $orange tests/verify_orange.py

.\.venv\Scripts\python.exe jury_2_dw/benchmark_models.py --train-cap 0 --val-cap 0 --test-cap 0

.\.venv\Scripts\python.exe jury_2_dw/mine_associations.py --sample-size 0

.\.venv\Scripts\python.exe jury_2_dw/clinical_profiling.py

.\.venv\Scripts\python.exe jury_1_dw/etl_and_olap.py

.\.venv\Scripts\python.exe jury_1_dw/sql.py

.\.venv\Scripts\python.exe jury_1_dw/export_tables.py

.\.venv\Scripts\python.exe -m pytest -q --junitxml=deliverables/logs/pytest.xml

.\.venv\Scripts\python.exe -m streamlit run jury_2_dw/app.py --server.headless true --server.address 127.0.0.1 --server.port 8512
--browser.gatherUsageStats false

.\.venv\Scripts\python.exe tests/browser_evidence.py

.\.venv\Scripts\python.exe tools/record_provenance.py
```

## Results

* Backup: 70 non-cache project files verified with SHA-256 against the external pre-edit backup. The dataset remains byte-identical.

* Full-data benchmark: completed, five candidates, 101,766 encounters, patient-separated train/validation/test. Random Forest selected on validation macro F1. Its holdout scores are in `best_model_report.json` and full execution output in `deliverables/logs/benchmark.log`.

* Mining: completed on all encounters, both algorithms yield 85 itemsets and 23 filtered rules. Complete outputs agree; an independent metric calculation is covered by tests.

* Profiling: completed on all encounters, covering five grouping dimensions. Group counts and early-readmission totals are verified for complete-cohort profiles.

* Warehouse: rebuilt from the preserved local CSV; 101,766 joined fact rows, unique keys and SQLite integrity pass. Roll-up, slice and table export commands completed.

* Pytest: **29 passed**, zero final failures. Includes seven diagnosis cases, feature/missing handling, patient separation, all five algorithms on lightweight data, saved artifact inference, mining metrics, profiles, warehouse sample/full integrity, all six Jury 2 views, missing/corrupt result handling, Jury 1 dashboard, and execution from another working directory. The small-data training tests are smoke tests; the separately logged benchmark is the full-data run.

* Streamlit: startup health endpoint returned HTTP 200. Browser checks passed for Association Mining, Model Benchmark, Best Model, Clinical Profiling and Live Risk Studio, including algorithm switching, a lift-slider input, metric selection, stay-profile selection and a saved-model prediction. Overview is tested by AppTest and initial browser loading. Browser page-error list is empty.

* Screenshots: genuine Chrome viewport captures, inspected visually, in `deliverables/evidence`. They show actual generated results and working controls.

* Orange: XML/classes/channels/domains checked; 17 discrete association inputs, 77 itemsets, 45 output rules. Four configured learners pass supplementary five-fold evaluation; Confusion Matrix, ROC, Distributions and Rule Table receive real inputs. Desktop rule output and final Test & Score are inspected separately. Results use 3,000 unique patients and are not the Python benchmark scores.

## Failures encountered and corrections

1. Initial Orange workflows had no saved dataset selection. Correct `.tab` files loaded as discrete, while default Iris reproduced the reported message. Explicit File settings repair the workflow. This is a verified failure mechanism; the original user's pre-edit File selection could not be recovered from an unsaved session.

2. Orange desktop classification exposed a failed Tree learner with its default binary categorical splitting. The generator now disables exhaustive binary splitting and sets bounded depth/minimum sample settings. The verification harness now reads the saved workflow settings, and the revised real-widget/CV checks pass.

3. The first pytest run had 27 passes and two failures: a test wrongly assumed SQL row order, and a subprocess ran out of memory while application test caches accumulated. The key comparison now uses a set and test caches are released between tests. The subsequent run passed all 29 cases. Initial failure output is retained as `pytest_initial.log`.

4. A pandas downcasting warning was removed by replacing marker conversion with a mask. The subsequent suite emitted no warning summary.

5. Port 8501 was occupied. A separate verification app used 8512; the existing process was left alone.

6. Playwright's browser download failed. The browser test uses installed Google Chrome instead. Download output remains in `browser_install.log`.

7. The initial browser radio action targeted a hidden native input intercepted by Streamlit's visual label. The test now clicks the visible label, and the full interaction run passes. The failure is retained in `browser_radio_failure.log`.

8. The app closure stopped the verification server and in-progress browser work. The server was restarted, its health checked, and browser evidence recaptured. A startup race produced one connection-refused attempt before the server was ready; the completed later run is recorded in `browser.json`.

9. An initial provenance serialization rejected XGBoost's NaN missing-value parameter; the recorder explicitly serializes that configuration as the string `NaN`. It does not alter the model or its predictions.

10. Running a second Orange Python process alongside the desktop instance reached the machine's virtual-memory limit and caused import/paging-file failures. The temporary desktop instance opened for verification was closed after capturing evidence, then Orange validation was rerun on its own. No operating-system memory settings were changed. Avoid running several Orange and model sessions simultaneously on this machine.

## Warnings and limits

Orange widgets emit upstream deferred-method decoration warnings (Associate commit and learner apply methods), recorded in `orange.log`. Rule output, learner evaluation and table rendering pass despite these warnings. Do not suppress them and claim a warning-free Orange runtime.

The PDFs and their render/text/layout validation are recorded by `tools/validate_deliverables.py` in `deliverables/logs/pdf_validation.json`. The final Git state and complete file list are recorded in `FINAL_COMPLETION_REPORT.md` and `FINAL_TREE.txt`.

# Chapter 13 - Results and Discussion

## 13.1 Consolidated findings

The integrated system is reproducible across warehouse construction, model comparison, rule mining and descriptive profiling. The active warehouse retains all 101,766 encounters, and a full fact/dimension join preserves the same count. The primary dataset remains byte-identical to the audit copy. The three required Streamlit views render actual generated results and support the tested interactions.

Random Forest wins the declared validation objective and generalises similarly in macro F1 to the untouched patient test set. Its test macro F1 of 0.43302 remains modest, and early-readmission recall of 0.30038 is insufficient for a reliable patient-care workflow. The majority baseline comparison demonstrates why reporting only accuracy would be misleading.

Both association algorithms return matching results on the same baskets and thresholds. The strongest retained lift is about 1.229, so the patterns are weak-to-moderate enrichments within the observed prescribing data. Clinical profiles highlight differences across utilisation cohorts, particularly prior inpatient history, without establishing causation. Orange demonstrates valid categorical workflows but its different sample and representation must remain clearly labelled.

## 13.2 Limitations and threats to validity

The implementation uses one historical dataset and one fixed patient-group split. It does not estimate uncertainty across repeated splits, test temporal drift, validate on a separate healthcare system, calibrate probabilities or quantify performance by demographic subgroups. Candidate hyperparameters were fixed rather than exhaustively tuned, and the Naive Bayes assumptions are poorly matched to the feature representation.

Repeated encounters remain in descriptive mining and profiling, making them encounter-weighted rather than independent patient observations. Diagnosis grouping simplifies clinical detail. A 250-row permutation sample is intentionally lightweight and noisy. Preserved BI assets were not interactively retested in their desktop applications. These limitations constrain claims and identify concrete next validation tasks.

# Chapter 14 - Conclusion and Future Scope

## 14.1 Work completed

Jury 1 functionality is preserved in a repaired local-source ETL/warehouse flow. Jury 2 provides five comparable classifiers, validation-based selection, a saved multiclass inference pipeline, matching Apriori/FP-Growth results, five clinical profiles and the three required dashboard views. A recoverable backup, dependency records, tests, genuine screenshots, technical handover and viva material support team transfer and academic review.

The academic contribution is an auditable end-to-end analytical prototype whose displayed results can be traced to a source cohort and executed code. Honest reporting of limited classification performance and observational rule meaning is part of the result. The deliverable is prepared for demonstration, with explicit boundaries on what was and was not tested.

## 14.2 Roadmap

Future modelling work should prioritise repeated patient-group validation, temporal and external testing, probability calibration and subgroup error analysis. Feature improvements and alternative class-imbalance strategies should be evaluated with a declared objective and an untouched final test cohort. Association studies should examine stability across cohorts, patient-level sensitivity analyses and clinically reviewed definitions.

Operational intelligence requires time-stamped admission/discharge and capacity data, appropriate occupancy forecasting baselines, capacity constraints and validation against real operational outcomes. A separate length-of-stay prediction task and a bed-capacity forecasting model could then be built and evaluated. Those features remain future scope, despite their presence in the broader project title. Clinical deployment would require a substantially different validation and governance programme.

# References

[1] UCI Machine Learning Repository. Diabetes 130-US Hospitals for Years 1999-2008. Official dataset description and provenance.

https://archive.ics.uci.edu/dataset/296/diabetes+130-us+hospitals+for+years+1999-2008

[2] scikit-learn documentation. Metrics and scoring: quantifying the quality of predictions; pipelines, grouping and model documentation.

https://scikit-learn.org/stable/modules/model_evaluation.html

[3] mlxtend documentation. Association rules: support, confidence, lift and frequent-pattern interfaces.

https://rasbt.github.io/mlxtend/user_guide/frequent_patterns/association_rules/

[4] Orange Data Mining documentation. File widget and native three-header tab-separated format.

https://orangedatamining.com/widget-catalog/data/file/

[5] Streamlit documentation. App testing framework and AppTest.

https://docs.streamlit.io/develop/api-reference/app-testing

[6] XGBoost documentation. Python estimator and tree-boosting parameters.

https://xgboost.readthedocs.io/en/stable/parameter.html

References accessed 21 October 2026. Library versions and the source SHA-256 are recorded in run_provenance.json. The implementation findings in this report derive from local source code, executed logs, CSV/JSON results and genuine screenshots, rather than unsupported literature claims.

# Appendix A - Reproduction and artifact index

Run commands and dependency instructions are in README.md. Technical handover, viva preparation, Orange demonstration and validation records are in docs/. Current analysis outputs are under jury_2_dw/outputs. The external pre-edit backup and the local archive preserve historical work. The complete final file tree, change disposition and Git state are in docs/FINAL_COMPLETION_REPORT.md.

| Artifact | Role |
|---|---|
| model_comparison.csv / .json | Validation metrics for five models |
| best_model.joblib | Complete fitted preprocessing/classifier bundle |
| best_model_report.json | Selected-model holdout metrics and per-class report |
| run_provenance.json | Source hash, dependency versions, seeds, candidate parameters and split hashes |
| apriori_* / fp_growth_* | Real itemsets, rule tables and measured timing summaries |
| profile_*.csv / .json / .png | Clinical cohort counts, rates and charts |
| deliverables/evidence/ | Genuine browser and Orange screenshots |
| deliverables/logs/ | Executed pipeline, test, browser, Orange and PDF validation records |